Like our previous sketches, these lines of code are the I/O pin numbers that we will use with our hardware. The first line tells the LiquidCrystal library which of the pins the LCD is connected to. The `backlight` pin is the built in LED on the LCD that we will fade in and out. Finally, `axisPin[]` is a numerical array that contains the three analog input pins used by the accelerometer.

```
const int threshold = 60;
const int brightness = 175;
```

These two lines are other configurable variables that determine how much movement needs to be detected before an answer is generated and the total brightness in the LCD backlight in the first and second line respectively.

```
int axis[3];
int oldAxis[3];
```

The last of our global variable declarations, `axis[]` and `oldAxis[]` are two arrays that we will use to keep track of which axis has been read and what the old values were the last time we read them. This will help to determine how much movement has been detected in shaking or rotating the device.

setup() and loop()

Our `setup()` function is fairly small:

```
lcd.begin(16, 2);
randomSeed(analogRead(A5));
getReading();
oldReading();
```

We begin with a function that starts up the LCD and tells it how big it is, in this case 16×2 characters. We then seed the random function with a reading from analog in pin A5 with nothing connected to it. Next, we take a quick reading from the accelerometer and set these as the old values using two of our functions, `getReading()` and `oldReading()`. We will look at these more after we talk about the `loop()` function.

```
getReading();
  if (abs(axis[0]-oldAxis[0]) > threshold ||
      abs(axis[1]-oldAxis[1]) > threshold ||
      abs(axis[2]-oldAxis[2]) > threshold) {
```

Because we have sufficiently compartmentalized our code using functions, the `loop()` function is also fairly compact. We begin with reading the sensor values first. The second, third, and fourth lines will check to see if our threshold has been exceeded on any axis by subtracting the old values from the newest values and testing the difference. We've used the `abs()` function to get the absolute value of this difference because we could have positive or negative values here, but we only want to know the difference. Using `abs()` saves us from having to also check if the old values are larger than the new values because it's only the amount the values change that we are really interested in.

```
getAnswer();
delay(500);
getReading();
oldReading();
```